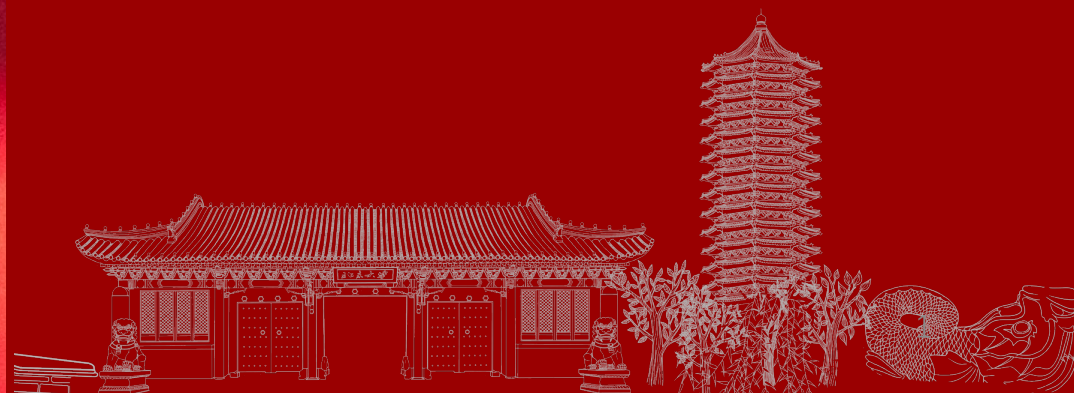


人工智能引论

15. 自然语言处理 -句法分析与分词

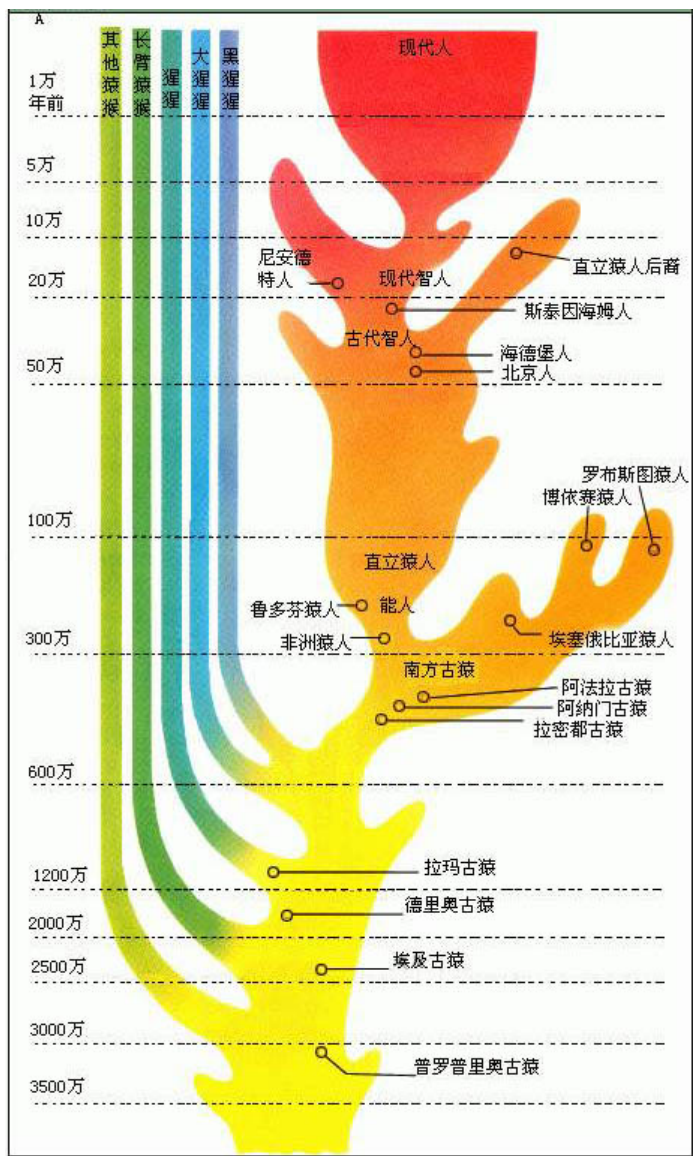
授课教师 张辉帅

2026年5月11日

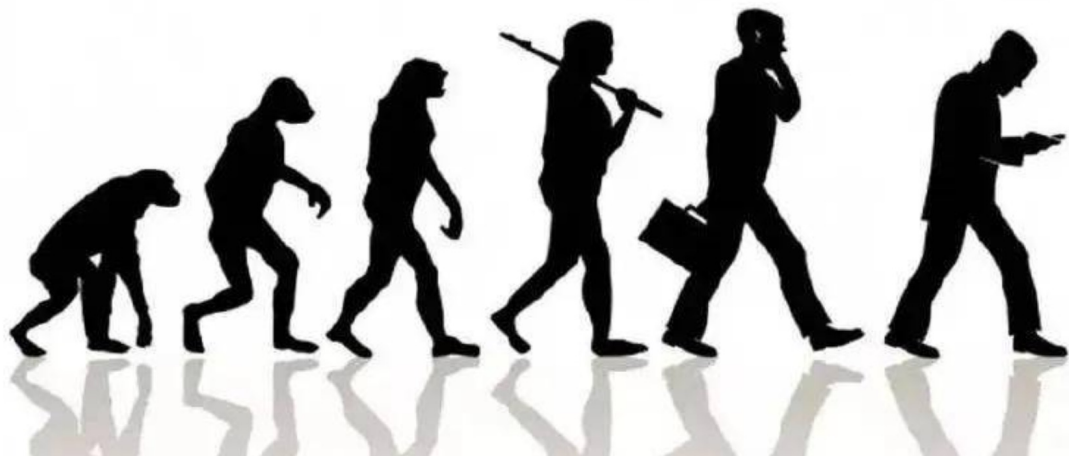


- 上下文无关的文法 (Context-Free Grammar)
- 句法分析 (Parsing)
- n元模型 (n-gram)
- 分词 (Tokenization)
- 基于马尔科夫模型的文本生成

背景介绍——语言的重要性



- 大约10万年前，人类学会说话
- 大约5千年前，人类学会写字
- 图灵测试：基于语言
- 语言具有普适性，并捕捉大量智能行为
- 语言和文字的交流促进了人类文明的发展



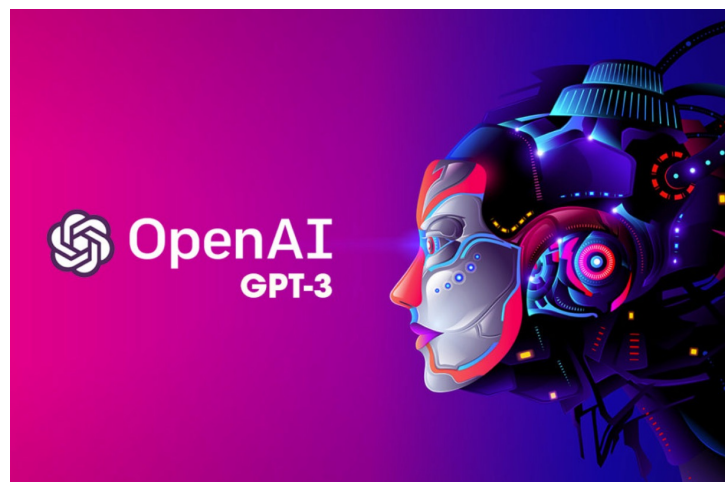
背景介绍——为何研究自然语言处理？

自然语言处理 (natural language processing, NLP)

- 让机器与人类交流
- 让机器学习到知识
- 推动对语言和语言使用的科学理解



2011, IBM Watson



2020, OpenAI GPT-3



2022, OpenAI ChatGPT

背景介绍——NLP可以做些什么？

- 语音识别 (speech recognition)
- 文本-语音合成 (text-to-speech synthesis)
- 机器翻译 (machine translation)
- 信息提取 (information extraction)
- 信息检索 (information retrieval)
- 问答 (question answering)
- 机器写作
- 文本摘要
-

Ameca多语言测试



2023年4月



背景介绍——NLP难点何在？

- **文法** (grammar) : 定义合法句的句法 (syntax)
- **语义规则** (semantic rule) : 定义其含义

自然语言无法像形式语言一样清晰地表示：

- 不同人在不同时间对语言的判断有区别
 - “Not to be invited is sad” ☒
 - “To be not invited is sad” ☐? ☒?
- 自然语言存在歧义，也是模糊不清的
 - “He saw her duck” -> 她的鸭子？她躲避？
 - “That’s great!” -> how? what?
- 自然语言没有正式定义从符号到对象的映射

句法 (Syntax)

- **句法**: 规定了如何用单词来组成句子

- "Just before 9:00 o'clock, Sherlock Holmes stepped briskly into the room." 😊

- "Just before Sherlock Holmes 9 o'clock stepped briskly the room." 😞

- **句法亦是有歧义的**

- "I saw the man on the mountain with a telescope"

语义 (Semantic)

- **语义**：语言包含的含义
 - 词汇语义 (lexical semantic)：单独词汇的含义
 - 组合语义 (Compositional semantics)：基于句法组合而成的含义
- 不同句子包含相同含义
 - "Just before nine o'clock Sherlock Holmes stepped briskly into the room." 😊
 - "Sherlock Holmes stepped briskly into the room just before nine o'clock." 😊
 - "A few minutes before nine, Sherlock Holmes walked quickly into the room." 😊
- 句法结构良好，却毫无意义
 - "Colorless green ideas sleep furiously." 😞

语义 (Semantic)

Los Angeles Times



CALIFORNIA



Big rig carrying fruit crashes on 210 Freeway, creates jam

By JOSEPH SERNA MAY 20, 2013 | 6:35 AM

Monday's morning commute started off horribly for drivers in the San Gabriel Valley when a big rig carrying fruit overturned on the 210, blocking lanes in both directions in Monrovia for most of the morning.

The big rig crashed through the center divider just before 5 a.m. near Myrtle Avenue. Three westbound lanes and two eastbound lanes will be blocked until about 9:15 a.m., according to the California Highway Patrol.

顾客: “豆腐多少钱?”

老板: “两块。”

顾客: “两块一块啊?”

老板: “一块。”

顾客: “一块两块啊?”

老板: “两块。”

顾客: “到底是两块一块, 还是一块两块,”

老板: “是两块一块

顾客: 那就是五毛一块呗!

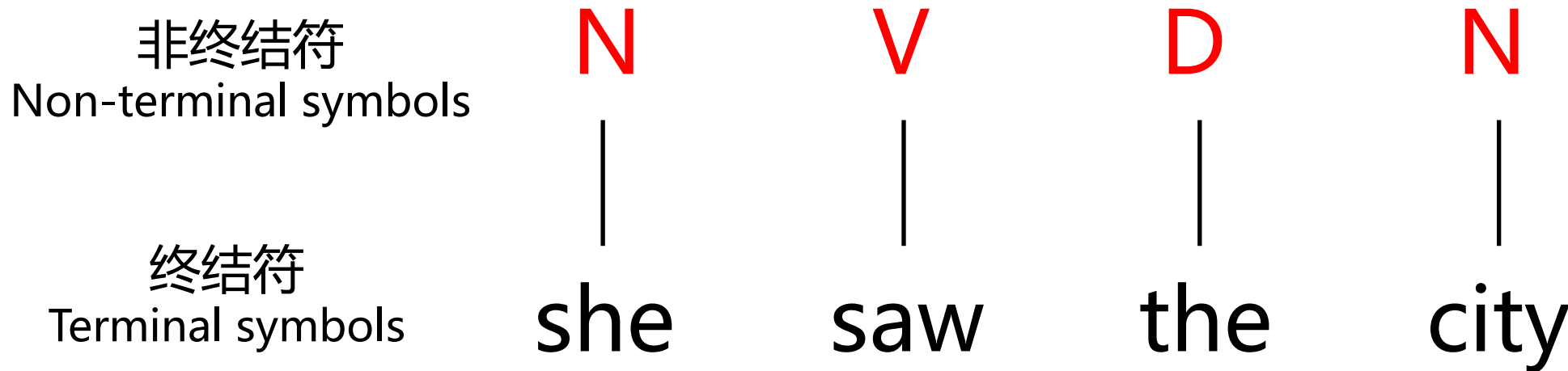
老板: 去你的, 不卖你了! 都把老子整糊涂了!

上下文无关的文法 (Context-Free Grammar)



北京大学
PEKING UNIVERSITY

- **文法** (grammar) 是一组规则, 定义了合法短语的树结构
- **语言** (language) 是遵循这些规则的句子集



上下文无关的文法 (Context-Free Grammar)



N → she | city | car | Harry | ...

D → the | a | an | ...

V → saw | ate | walked | ...

P → to | on | over | ...

ADJ → blue | busy | old | ...

上下文无关的文法 (Context-Free Grammar)

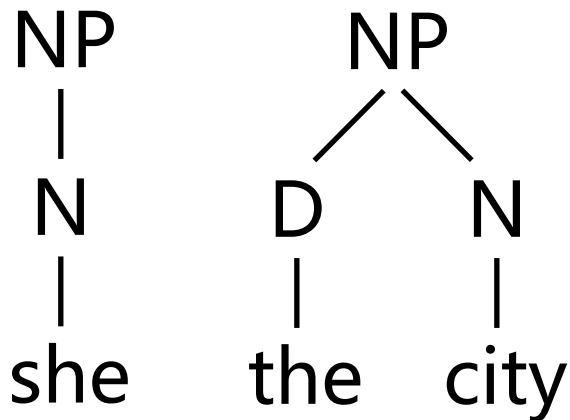


北京大学
PEKING UNIVERSITY

名词短语

Noun Phrase

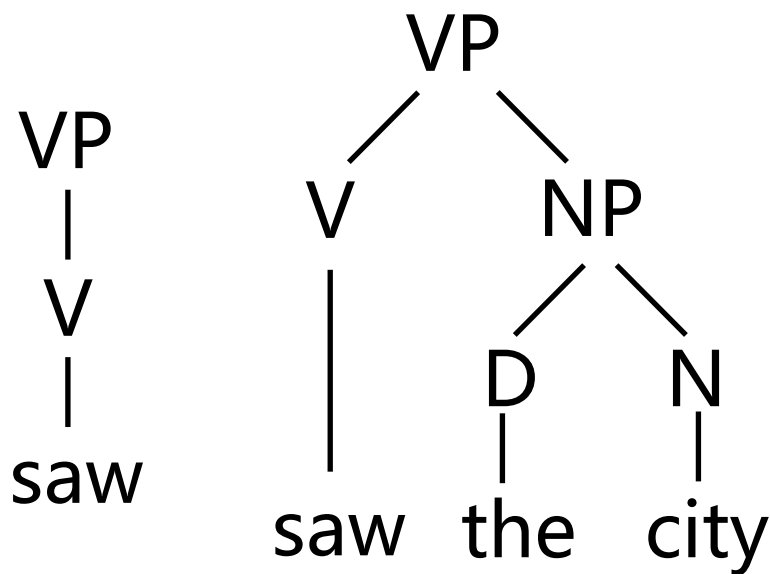
NP $\rightarrow$ N | D N



动词短语

Verb Phrase

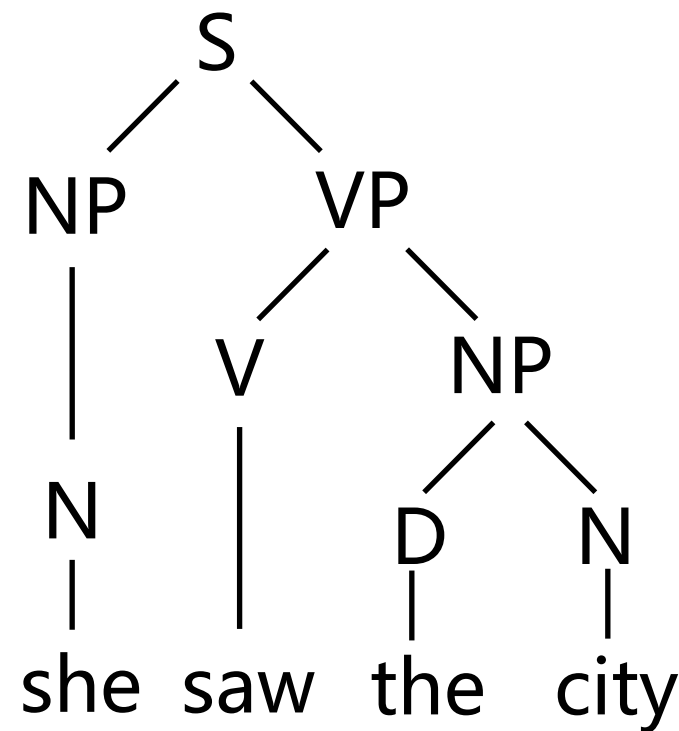
VP $\rightarrow$ V | V NP



句子

Sentence

S $\rightarrow$ NP VP



she saw the city

上下文无关的文法 (Context-Free Grammar)



概率上下文无关的文法 (Probabilistic Context-Free Grammar)

ϵ_0 的文法(每个规则有个示例短语)

- **句法范畴** (syntactic category) : 句子 (S)、名词短语 (NP)、动词短语 (VP)、形容词列表 (Adjs)、介词短语 (PP) 和关系从句 (RelClause)

$S \rightarrow NP VP$	[0.90]	I+feel a breeze
$S Conj S$	[0.10]	I feel a breeze+and+It stinks
$NP \rightarrow Pronoun$	[0.25]	I
$Name$	[0.10]	Ali
$Noun$	[0.10]	pits
$Article Noun$	[0.25]	the+wumpus
$Article Adjs Noun$	[0.05]	the+smelly dead+wumpus
$Digit Digit$	[0.05]	3 4
$NP PP$	[0.10]	the wumpus+in 1 3
$NP RelClause$	[0.05]	the wumpus+that is smelly
$NP Conj NP$	[0.05]	the wumpus+and+I
$VP \rightarrow Verb$	[0.40]	stinks
$VP NP$	[0.35]	feel+a breeze
$VP Adjective$	[0.05]	smells+dead
$VP PP$	[0.10]	is+in 1 3
$VP Adverb$	[0.10]	go+ahead
$Adjs \rightarrow Adjective$	[0.80]	smelly
$Adjective Adjs$	[0.20]	smelly+dead
$PP \rightarrow Prep NP$	[1.00]	to+the east
$RelClause \rightarrow RelPro VP$	[1.00]	that+is smelly

$Adjs \rightarrow Adjective$ [0.80] smelly
| $Adjective Adjs$ [0.20] smelly+dead

上下文无关的文法 (Context-Free Grammar)



概率上下文无关的文法 (Probabilistic Context-Free Grammar)

ε_0 的词典 (lexicon)

<i>Noun</i>	→	stench [0.05] breeze [0.10] wumpus [0.15] pits [0.05] ...
<i>Verb</i>	→	is [0.10] feel [0.10] smells [0.10] stinks [0.05] ...
<i>Adjective</i>	→	right [0.10] dead [0.05] smelly [0.02] breezy [0.02] ...
<i>Adverb</i>	→	here [0.05] ahead [0.05] nearby [0.02] ...
<i>Pronoun</i>	→	me [0.10] you [0.03] I [0.10] it [0.10] ...
<i>RelPro</i>	→	that [0.40] which [0.15] who [0.20] whom [0.02] ...
<i>Name</i>	→	Ali [0.01] Bo [0.01] Boston [0.01] ...
<i>Article</i>	→	the [0.40] a [0.30] an [0.10] every [0.05] ...
<i>Prep</i>	→	to [0.20] in [0.10] in [0.05] near [0.10] ...
<i>Conj</i>	→	and [0.50] or [0.10] but [0.20] yet [0.02] ...
<i>Digit</i>	→	0 [0.20] 1 [0.20] 2 [0.20] 3 [0.20] 4 [0.20] ...

- **开放类** (open class) :
名词、名称、动词、形容词、副词 (数量大、变化)
- **封闭类** (closed class) :
代词、关系代词、冠词、介词、连词 (数量小、几乎不变)

句法分析 (Parsing)

- 句法分析：根据语法规则分析一串单词以获得其短语结构的过程

使用 ϵ_0 的文法 对 “The wumpus is dead” 做句法分析

$S \rightarrow NP VP$
| $S Conj S$

$NP \rightarrow Pronoun$
| $Name$
| $Noun$
| $Article Noun$
| $Article Adjs Noun$
| $Digit Digit$
| $NP PP$
| $NP RelClause$
| $NP Conj NP$

$VP \rightarrow Verb$
| $VP NP$
| $VP Adjective$
| $VP PP$
| $VP Adverb$

$Adjs \rightarrow Adjective$
| $Adjective Adjs$

$PP \rightarrow Prep NP$

$RelClause \rightarrow RelPro VP$

自
顶
向
下

项目列表	规则
S	
$NP VP$	$S \rightarrow NP VP$
$NP VP Adjective$	$VP \rightarrow VP Adjective$
$NP Verb Adjective$	$VP \rightarrow Verb$
$NP Verb \mathbf{dead}$	$Adjective \rightarrow \mathbf{dead}$
$NP \mathbf{is dead}$	$Verb \rightarrow \mathbf{is}$
$Article Noun \mathbf{is dead}$	$NP \rightarrow Article Noun$
$Article \mathbf{wumpus is dead}$	$Noun \rightarrow \mathbf{wumpus}$
$\mathbf{the wumpus is dead}$	$Article \rightarrow \mathbf{the}$

自
底
向
上

缺点：低效、易陷入死循环！

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

——自底向上的图表句法分析算法

- 乔姆斯基范式 (CNF, Chomsky Normal Form)
 - ✓ $A \rightarrow B C$
 - ✓ $A \rightarrow a$
 - ✓ $S \rightarrow \emptyset$
- CNF语法都是二分叉的, 任何语法都可以转化为一个弱等价的CNF形式
 - ✓ 例1: $A \rightarrow B c$ 转化为 $A \rightarrow B C, C \rightarrow c$
 - ✓ 例2: $A \rightarrow B C D$ 转化为 $A \rightarrow B X, X \rightarrow C D$
 - ✓ **CYK处理的CFG必须是CNF形式的**

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

- 考虑以下CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

- 输入以下句子:

✓ $w = \text{baaba}$

步骤1: 构建三角形表格

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

Triangular table for the word 'baaba'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤2: 填充表格

由给定语法得知:

'b' 由变量B得到
将B填入第一行第一
列中, 也即:

$\text{'b'} = X(1,1) = \{B\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2					
1	{B}				
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

同样, 由给定语法知:

'a' 由变量A和C得到, 将A和C填入第一行第二列, 也即:

'a' : $X(2,2) = \{A, C\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2					
1	{B}	{A,C}			
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤2: 填充表格

同理:

'a' : $X(3,3) = \{A, C\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2					
1	{B}	{A,C}	{A,C}		
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

同理:

'b' : $X(4,4) = \{B\}$

'a' : $X(5,5) = \{A,C\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2					
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

第二行考虑长度为2的
子串 'ba', 'aa', 'ab', 'ba':

'ba': $X(1,2)$

$= (X(1,1), X(2,2))$

$= \{B\}\{A, C\} = \{BA, BC\}$

$= \{A, S\} = \{S, A\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2	{S,A}				
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

'aa': $X(2,3)$

$X(i,j) = (X(i,i), X(i+1,j))$

$= (X(2,2), X(3,3))$

$= \{A, C\} \{A, C\}$

$= \{AA, AC, CA, CC\}$

$= \{AA, AC, CA, CC\} = \{B\}$

5	$X(1,5)$				
4	$X(1,4)$	$X(2,5)$			
3	$X(1,3)$	$X(2,4)$	$X(3,5)$		
2	$X(1,2)$	$X(2,3)$	$X(3,4)$	$X(4,5)$	
1	$X(1,1)$	$X(2,2)$	$X(3,3)$	$X(4,4)$	$X(5,5)$
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2	{S,A}	{B}			
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

类似地:

'ab': $X(3,4)$

$= \{AB, CB\} = \{AB, \text{CB}\}$

$= \{S, C\}$

'ba': $X(4,5)$

$= \{BA, BC\} = \{S, A\}$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4					
3					
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = baaba$

步骤2: 填充表格

第三行考虑长度为3的子

串 'baa', 'aab', 'aba':

'baa': $X(1,3)$

两种可能: ['b','aa']和['ba','a']

['b','aa'] = $\{B\}\{B\}$

['ba','a'] = $\{S,A\}\{A,C\}$

$\{BB\} \cup \{SA, SC, AA, AC\}$

$= \{BB, SA, SC, AA, AC\} = \{\emptyset\}$

5	$X(1,5)$				
4	$X(1,4)$	$X(2,5)$			
3	$X(1,3)$	$X(2,4)$	$X(3,5)$		
2	$X(1,2)$	$X(2,3)$	$X(3,4)$	$X(4,5)$	
1	$X(1,1)$	$X(2,2)$	$X(3,3)$	$X(4,4)$	$X(5,5)$
	'b'	'a'	'a'	'b'	'a'

5					
4					
3	$\{\emptyset\}$	$\{B\}$	$\{B\}$		
2	$\{S,A\}$	$\{B\}$	$\{S,C\}$	$\{S,A\}$	
1	$\{B\}$	$\{A,C\}$	$\{A,C\}$	$\{B\}$	$\{A,C\}$
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤2: 填充表格

第四行考虑长度为4
的子串'baab', 'aaba':

'baab'三种组合:

['b', 'aab'], ['ba', 'ab'],

['baa', 'b']

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4	{ $\emptyset$ }	{S,A,C}			
3	{ $\emptyset$ }	{B}	{B}		
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤2: 填充表格

第四行考虑长度为4
的子串'baab', 'aaba':

'aaba'三种组合:

['a', 'aba'], ['aa', 'ba'],

['aab', 'a']

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5					
4	{ $\emptyset$ }	{S,A,C}			
3	{ $\emptyset$ }	{B}	{B}		
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤2: 填充表格

第五行考虑长度为5

的子串'baaba':

四种组合:

$['b', 'aaba'], ['baab', 'a'],$

$['ba', 'aba'], ['baa', 'ba']$

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

5	{S,A,C}				
4	{ $\emptyset$ }	{S,A,C}			
3	{ $\emptyset$ }	{B}	{B}		
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali **C**ocke, Daniel **Y**ounger, Tadeo **K**asami)

• CNF语法G:

✓ $S \rightarrow AB \mid BC$

✓ $A \rightarrow BA \mid a$

✓ $B \rightarrow CC \mid b$

✓ $C \rightarrow AB \mid a$

• 输入句子:

$w = \text{baaba}$

步骤3: 检查表格

$X(1,5) = \{S, A, C\}$

S在最后一个格子中

说明:

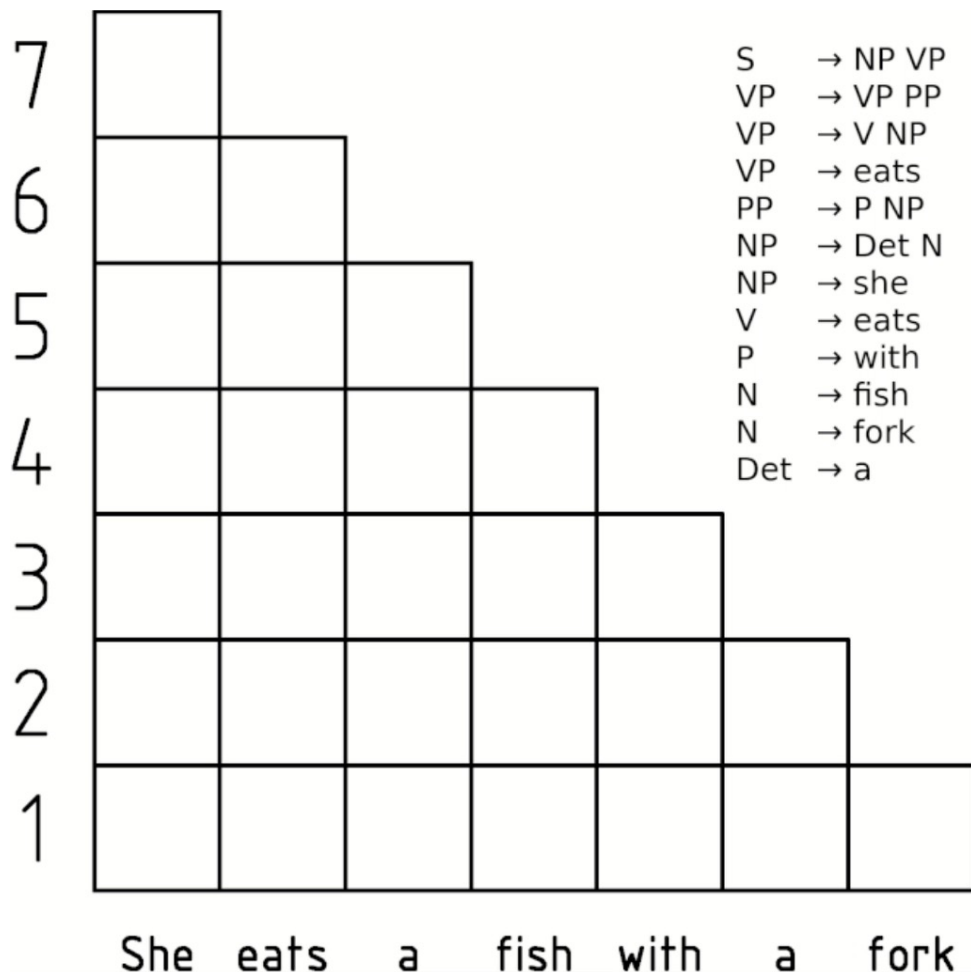
w满足语法L(G)

5	X(1,5)				
4	X(1,4)	X(2,5)			
3	X(1,3)	X(2,4)	X(3,5)		
2	X(1,2)	X(2,3)	X(3,4)	X(4,5)	
1	X(1,1)	X(2,2)	X(3,3)	X(4,4)	X(5,5)
	'b'	'a'	'a'	'b'	'a'

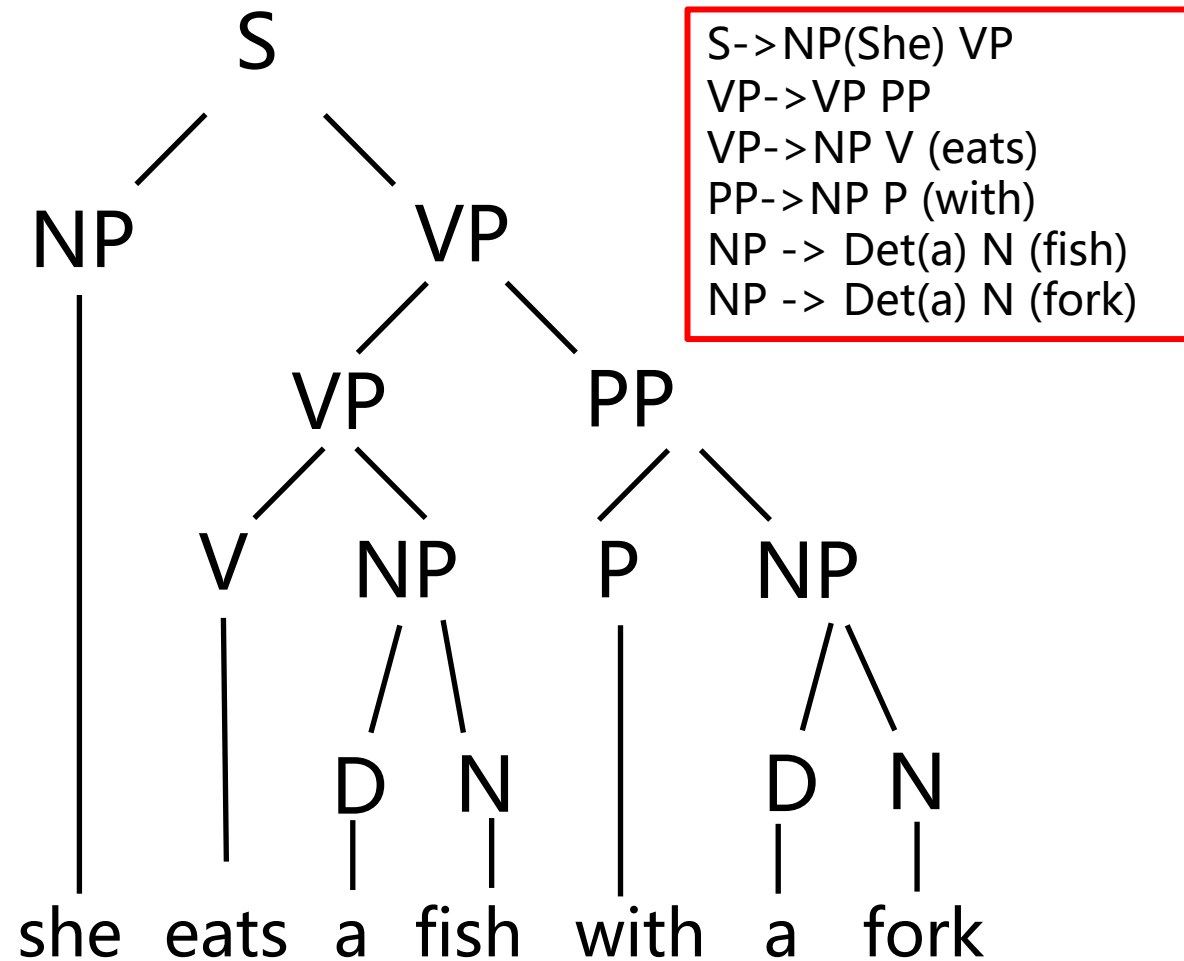
5	{S,A,C}				
4	{ $\emptyset$ }	{S,A,C}			
3	{ $\emptyset$ }	{B}	{B}		
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

句法分析 (Parsing)

CYK算法 (作者: Ali Cocke, Daniel Younger, Tadeo Kasami)



Sentence parsing using the CYK algorithm



S → NP(She) VP
VP → VP PP
VP → NP V (eats)
PP → NP P (with)
NP → Det(a) N (fish)
NP → Det(a) N (fork)

句法分析 (Parsing)

CYK算法 (作者: Ali Cocke, Daniel Younger, Tadeo Kasami)

```
let the input be a string  $I$  consisting of  $n$  characters:  $a_1 \dots a_n$   
let the grammar contain  $r$  nonterminal symbols  $R_1 \dots R_r$ , with start symbol  $R_1$ .  
let  $P[n, n, r]$  be an array of booleans. Initialize all elements of  $P$  to false.  
let  $back[n, n, r]$  be an array of lists of backpointing triples. Initialize all elements of  $back$  to the empty list.
```

n 个单词
 r 个非终结符

```
for each  $s = 1$  to  $n$   
  for each unit production  $R_v \rightarrow a_s$   
    set  $P[1, s, v] = \text{true}$ 
```

给三角表格的第一行赋值

```
for each  $l = 2$  to  $n$  -- Length of span  
  for each  $s = 1$  to  $n-l+1$  -- Start of span  
    for each  $p = 1$  to  $l-1$  -- Partition of span  
      for each production  $R_a \rightarrow R_b R_c$   
        if  $P[p, s, b]$  and  $P[l-p, s+p, c]$  then  
          set  $P[l, s, a] = \text{true}$ ,  
          append  $\langle p, b, c \rangle$  to  $back[l, s, a]$ 
```

给三角表格的其他行赋值

```
if  $P[n, 1, 1]$  is true then  
   $I$  is member of language  
  return  $back$  -- by retracing the steps through  $back$ , one can easily construct all possible parse trees of the string.  
else  
  return "not a member of language"
```

句法分析 (Parsing)

CYK算法 (作者: Ali Cocke, Daniel Younger, Tadeo Kasami)

——自底向上的图表句法分析算法的概率版本【只需了解】

function CYK-PARSE(*words*, *grammar*) **returns** 一个句法分析树表

inputs: *words*, 单词列表

grammar, LEXICALRULES和GRAMMARRULES结构

$T \leftarrow$ 一个表 // $T[X, i, k]$ 是涵盖 $words_{i:k}$ 的最可能的 X 树

$P \leftarrow$ 一个表, 初始时全为0 // $P[X, i, k]$ 表示树 $T[X, i, k]$ 的概率

// 为每个单词插入词汇范畴

for $i = 1$ **to** LEN(*words*) **do**

for each (X, p) **in** *grammar*.LEXICALRULES($words_i$) **do**

$P[X, i, i] \leftarrow p$

$T[X, i, i] \leftarrow \text{Tree}(X, words_i)$

// 通过 $Y_{ij} + Z_{j+1:k}$ 构建 $X_{i:k}$, 最短区间优先

for each (i, j, k) **in** SUBSPANS(LEN(*words*)) **do**

for each (X, Y, Z, p) **in** *grammar*.GRAMMARRULES **do**

$PYZ \leftarrow P[Y, i, j] \times P[Z, j+1, k] \times p$

if $PYZ > P[X, i, k]$ **do**

$P[X, i, k] \leftarrow PYZ$

$T[X, i, k] \leftarrow \text{Tree}(X, T[Y, i, j], T[Z, j+1, k])$

return T

表格P中元素的值
由布尔值(0,1)变为概率值

- 空间复杂度 $O(n^2m)$

- 时间复杂度 $O(n^3m)$

- n : 句子中单词的数量

- m : 文法中非终结符的数量

function SUBSPANS(N) **yields** (i, j, k)元组

for $length = 2$ **to** N **do**

for $i = 1$ **to** $N + 1 - length$ **do**

$k \leftarrow i + length - 1$

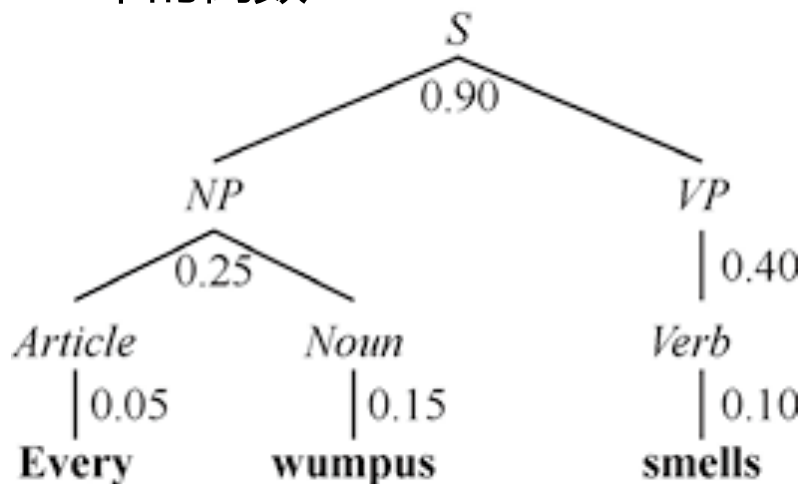
for $j = i$ **to** $k - 1$ **do**

yield (i, j, k)

句法分析 (Parsing)

- **A*搜索**比CYK算法快, 比 $O(n)$ 慢【只需了解】

- 每个状态都是一个项目 (单词或范畴) 列表
- 初始状态: 单词列表
- 目标状态: S
- 状态代价: 目前为止应用的规则所定义的概率的倒数



步骤	状态 (项目列表)
0	[every, wumpus, smells]
1	[Article, Noun, Verb] (查词典, 单词 → 词类)
2	[NP, Verb] (用 NP → Article Noun 归约前两项)
3	[NP, VP] (用 VP → Verb 归约第三项)
4	[S] (用 S → NP VP 归约, 到达目标) ✓

整个句法分析树的概率:

$$0.9 \times 0.25 \times 0.05 \times 0.15 \times 0.40 \times 0.10 = 0.0000675$$

线性形式:

[S [NP [Article **every**][Noun **wumpus**]][VP [Verb **smells**]]]

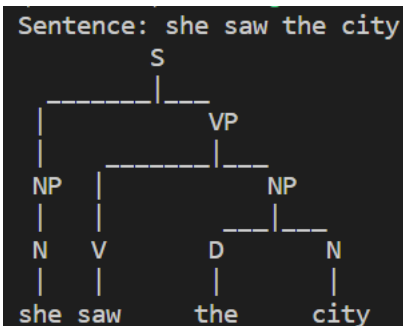
nlTK (Natural Language Toolkit)

- <https://www.nltk.org/>
- NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries
- NLTK has been called “a wonderful tool for teaching, and working in, computational linguistics using Python,” and “an amazing library to play with natural language.”

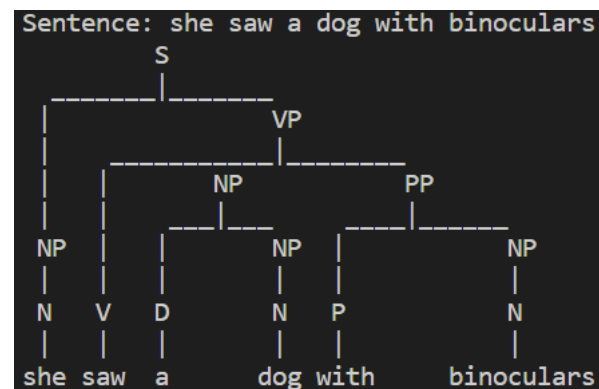
nltk (Natural Language Toolkit)



```
AI-NLP > code > cfg > cfg0.py > ...
1  import nltk
2
3  grammar = nltk.CFG.fromstring("""
4      S -> NP VP
5
6      NP -> D N | N
7      VP -> V | V NP
8
9      D -> "the" | "a"
10     N -> "she" | "city" | "car"
11     V -> "saw" | "walked"
12 """)
13
14 parser = nltk.ChartParser(grammar)
15
16 sentence = input("Sentence: ").split()
17 try:
18     for tree in parser.parse(sentence):
19         tree.pretty_print()
20         # tree.draw()
21 except ValueError:
22     print("No parse tree possible.")
```



```
AI-NLP > code > cfg > cfg1.py > ...
1  import nltk
2
3  grammar = nltk.CFG.fromstring("""
4      S -> NP VP
5
6      AP -> A | A AP
7      NP -> N | D NP | AP NP | N PP
8      PP -> P NP
9      VP -> V | V NP | V NP PP
10
11     A -> "big" | "blue" | "small" | "dry" | "wide"
12     D -> "the" | "a" | "an"
13     N -> "she" | "city" | "car" | "street" | "dog" | "binoculars"
14     P -> "on" | "over" | "before" | "below" | "with"
15     V -> "saw" | "walked"
16 """)
17
18 parser = nltk.ChartParser(grammar)
19
20 sentence = input("Sentence: ").split()
21 try:
22     for tree in parser.parse(sentence):
23         tree.pretty_print()
24         #tree.draw()
25         break
26 except ValueError:
27     print("No parse tree possible.")
```



n元模型 (n-gram)

- **n元**: 长度为n的**书面符号** (单词、字符、音节等) 序列
- 字符n元、单词n元...
- **一元**(unigram/1-gram)、**二元**(bigram/2-gram)、**三元**(trigram/3-gram)...
- “How often have I said to you that when you have eliminated the impossible whatever remains, however improbable, must be the truth?”
- **三元**(3-grams): 样例文本中在相邻连续序列中的三个单词
 - How often have
 - often have I
 - have I said
 -

n元模型 (n-gram)

- **n元**: 长度为n的**书面符号** (单词、字符、音节等) 序列
- 字符n元、单词n元...
- **一元**(unigram/1-gram)、**二元**(bigram/2-gram)、**三元**(trigram/3-gram)...
- “How often have I said to you that when you have eliminated the impossible whatever remains, however improbable, must be the truth?”
- **三元**(3-grams): 样例文本中在相邻连续序列中的三个单词
 - How often have
 - often have I
 - have I said
 -

n元模型 (n-gram)

- **n元**: 长度为n的**书面符号** (单词、字符、音节等) 序列
- 字符n元、单词n元...
- **一元**(unigram/1-gram)、**二元**(bigram/2-gram)、**三元**(trigram/3-gram)...
- “How often have I said to you that when you have eliminated the impossible whatever remains, however improbable, must be the truth?”
- **三元**(3-grams): 样例文本中在相邻连续序列中的三个单词
 - How often have
 - often have I
 - have I said
 -

标记化(分词) (Tokenization)

- 分词 (Word Tokenization)

- 将文本序列分割为独立的单词

- 例1: "Whatever remains, however improbable, must be the truth. "

- ❖ ["Whatever", "remains,", "however", "improbable,", "must", "be", "the", "truth."]

- ❖ ["Whatever", "remains", "however", "improbable", "must", "be", "the", "truth"]

分词 (Tokenization)

- 分词 (Word Tokenization)

- 将文本序列分割为独立的单词
- 例2: "Just before nine o'clock Sherlock Holmes stepped briskly into the room. "
- 例3: "He was dressed in a sombre yet rich style, in black frock-coat, shining hat, neat brown gaiters, and well-cut pearl-grey trousers. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例1: "I cannot waste time over this sort of fantastic talk, Sherlock. If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子

- 例1: "I cannot waste time over this sort of fantastic talk, Sherlock. If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子

- 例1: "I cannot waste time over this sort of fantastic talk, Sherlock. If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例2: "I cannot waste time over this sort of fantastic talk, Mr. Holmes. If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例2: "I cannot waste time over this sort of fantastic talk, Mr. Holmes.
If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例2: "I cannot waste time over this sort of fantastic talk, Mr. Holmes. If you can catch the man, catch him, and let me know when you have done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例2: "I cannot waste time over this sort of fantastic talk, Mr. Holmes.
If you can catch the man, catch him, and let me know when you have
done it. "

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子
- 例3: "I cannot waste time over this sort of fantastic talk, Mr. Holmes," he said. "If you can catch the man, catch him, and let me know when you have done it."

分词 (Tokenization)

- 句子切分(断句) (Sentence Tokenization)

- 将文本序列分割为独立的句子

- 例3: "I cannot waste time over this sort of fantastic talk, Mr. Holmes,"
he said. "If you can catch the man, catch him, and let me know when
you have done it."

- 中文在词与词之间没有任何空格之类的显示标志指示词的边界。因此，中文分词是很多自然语言处理系统中的**基础模块**和**首要环节**。

■ 基于字符串匹配的分词方法

- ✓ 按照一定的策略将待匹配的字符串和一个已建立好的“充分大的”词典中的词进行匹配，若找到某个词条，则说明匹配成功，识别了该词。

■ 基于机器学习的分词方法

- ✓ CRF(条件随机场)、HMM(隐马尔可夫)、SVM、深度学习(BiLSTM-CRF)等。
- ✓ 例：CRF：对汉字进行标注训练，不仅考虑了词语出现的频率，还考虑上下文。

■ 常用的中文分词工具包：

- ✓ jieba分词、SnowNLP、THULAC、NLPIR 等。

中文分词

```
import jieba
u="我来到中国北京大学"
#全模式
test1 = jieba.cut(u, cut_all=True)
print("全模式: " + "| ".join(test1))
#精确模式
test2 = jieba.cut(u, cut_all=False)
print("精确模式: " + "| ".join(test2))
#搜索引擎模式
test3= jieba.cut_for_search(u)
print("搜索引擎模式:" + "| ".join(test3))
```

例1: "我来到中国北京大学"

全模式: 我| 来到| 中国| 北京| 北京大学| 大学
精确模式: 我| 来到| 中国| 北京大学
搜索引擎模式: 我| 来到| 中国| 北京| 大学| 北京大学

例2: "北大是所好大学"

全模式: 北大| 是| 所| 好| 大学
精确模式: 北大| 是| 所| 好| 大学
搜索引擎模式: 北大| 是| 所| 好| 大学

例3: "AI智能引论是门好课"

全模式: AI| 智能| 引论| 是| 门| 好| 课
精确模式: AI| 智能| 引论| 是| 门| 好| 课
搜索引擎模式: AI| 智能| 引论| 是| 门| 好| 课

例4: "人工智能引论是门好课"

全模式: 人工| 人工智能| 智能| 引论| 是| 门| 好| 课
精确模式: 人工智能| 引论| 是| 门| 好| 课
搜索引擎模式: 人工| 智能| 人工智能| 引论| 是| 门| 好| 课

分词与n元模型演示

1-gram:

python ngrams.py 1 holmes

```
9722: ('the',)
5036: ('i',)
4926: ('and',)
4525: ('to',)
4513: ('of',)
4395: ('a',)
3004: ('in',)
2998: ('that',)
2832: ('it',)
2616: ('was',)
```

3-gram:

```
80: ('it', 'was', 'a')
71: ('one', 'of', 'the')
65: ('i', 'think', 'that')
59: ('out', 'of', 'the')
55: ('that', 'he', 'had')
55: ('there', 'was', 'a')
55: ('that', 'he', 'was')
55: ('that', 'it', 'was')
52: ('it', 'is', 'a')
49: ('that', 'i', 'have')
```

2-gram:

python ngrams.py 2 holmes

```
1158: ('of', 'the')
879: ('in', 'the')
521: ('it', 'was')
498: ('to', 'the')
463: ('it', 'is')
457: ('i', 'have')
405: ('that', 'i')
378: ('at', 'the')
370: ('and', 'i')
332: ('and', 'the')
```

4-gram:

```
24: ('i', 'have', 'no', 'doubt')
24: ('i', 'think', 'that', 'i')
20: ('i', 'do', 'n't', 'know')
19: ('it', 'seemed', 'to', 'me')
16: ('have', 'no', 'doubt', 'that')
16: ('i', 'do', 'not', 'know')
16: ('on', 'the', 'other', 'hand')
16: ('i', 'should', 'like', 'to')
15: ('to', 'me', 'to', 'be')
14: ('of', 'one', 'of', 'the')
```

```
1 from collections import Counter
2 import math
3 import nltk
4 import os
5 import sys
6
7 def main():
8     """Calculate top term frequencies for a corpus of documents
9     if len(sys.argv) != 3:
10         sys.exit("Usage: python tfidf.py n corpus")
11     print("Loading data...")
12     n = int(sys.argv[1])
13     corpus = load_data(sys.argv[2])
14
15     # Compute n-grams
16     ngrams = Counter(nltk.ngrams(corpus, n))
17
18     # Print most common n-grams
19     for ngram, freq in ngrams.most_common(10):
20         print(f"{freq}: {ngram}")
21
22 def load_data(directory):
23     contents = []
24     # Read all files and extract words
25     for filename in os.listdir(directory):
26         with open(os.path.join(directory, filename)) as f:
27             contents.extend([
28                 word.lower() for word in
29                 nltk.word_tokenize(f.read())
30                 if any(c.isalpha() for c in word)
31             ])
32     return contents
33
34 if __name__ == "__main__":
35     main()
```

大语言模型中的Tokenizer



- 子词 (subword) 切分
 - 平衡词表大小和序列长度
- 算法: Byte pair encoding(BPE)
- 是大模型设计的核心问题之一
- 可视化tokenizer:
 - <https://tiktokenizer.vercel.app>

Tiktokenizer

```
class Block(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.attn = CausalSelfAttention(config, layer_idx)
        self.mlp = MLP(config)

    def forward(self, x, ve, cos_sin, window_size):
        x = x + self.attn(norm(x), ve, cos_sin, window_size)
        x = x + self.mlp(norm(x))
        return x
```

gpt2

Token count
362

```
class MLP(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd, bias=False)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd, bias=False)
```

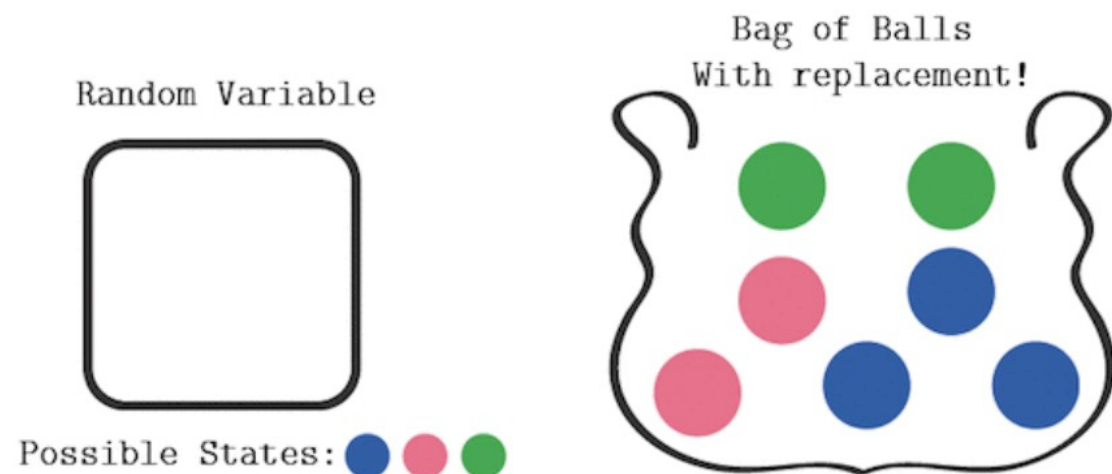
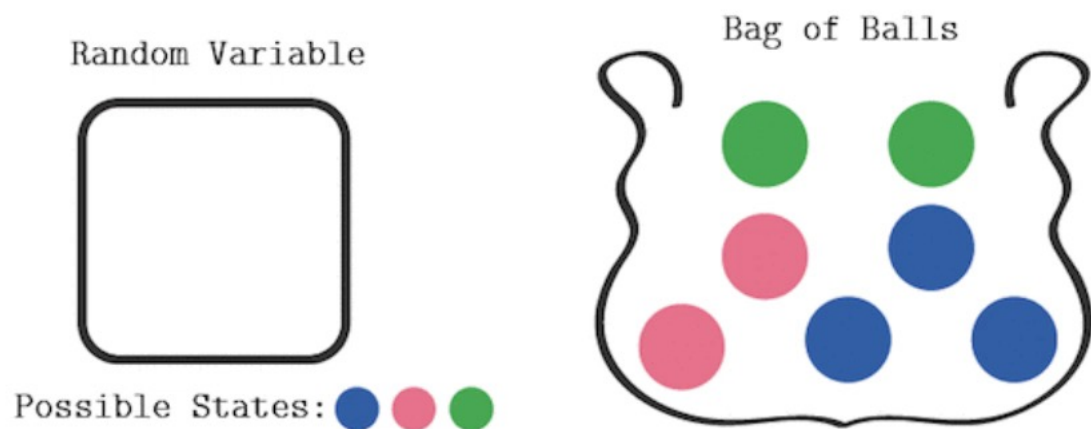
```
    def forward(self, x):
        x = self.c_fc(x)
        x = F.relu(x).square()
        x = self.c_proj(x)
        return x
```

```
class Block(nn.Module):
    def __init__(self, config, layer_idx):
        super().__init__()
        self.attn = CausalSelfAttention(config, layer_idx)
        self.mlp = MLP(config)
```

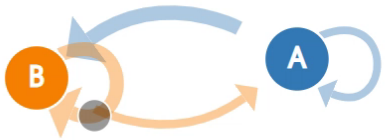
```
    def forward(self, x, ve, cos_sin, window_size):
        x = x + self.attn(norm(x), ve, cos_sin, window_size)
        x = x + self.mlp(norm(x))
        return x
```

马尔科夫模型 (Markov Model)

- **Markov链**具有**无后效性**：达到一个状态后, 决策只与当前状态有关, 而与以前的历史状态无关。猜猜下面哪个是Markov链?

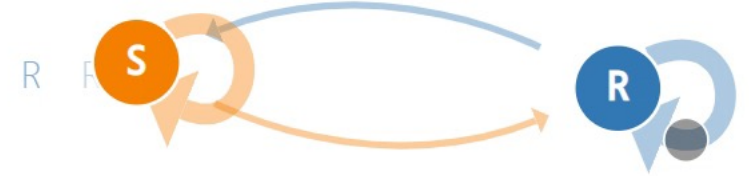


马尔科夫模型 (Markov Model)

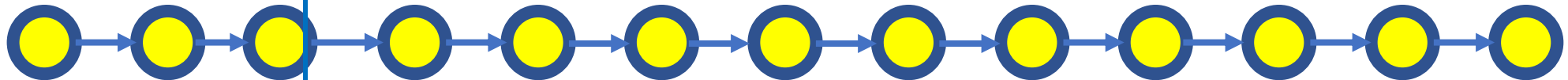


	A	B
A	$P(A A): 0.22$ 	$P(B A): 0.78$
B	$P(A B): 0.27$ 	$P(B B): 0.73$

:RRRRRRRRRRRRRRR



Were it not



马尔科夫模型 (Markov Model)

基于马尔科夫模型的文本生成

```
1  import markovify
2  import sys
3
4  # Read text from file
5  if len(sys.argv) != 2:
6      sys.exit("Usage: python generator.py sample.txt")
7  with open(sys.argv[1]) as f:
8      text = f.read()
9
10 # Train model
11 text_model = markovify.Text(text)
12
13 # Generate sentences
14 print()
15 for i in range(5):
16     print(text_model.make_sentence())
17     print()
18
```

How say you shall.

Ay, good my lord, if you'll prove perjurd if you will, I will, my lord.

O, no; for she hath a leak, And she whom all men depart.

Were it not news to hear That things ill which are not he.

I do but wish and full resolv'd, Foul-spoken coward, that thund'rest with thy case, thy habit, Wrench awe from fools, and children are even now out at gates; come!

Thou lov'st me well.

A preliminary version may often be posted for suggestion, comment and editing by those men Upon whose grave thou vow'dst pure chastity.

My lord, I have veil'd my look, I turn my back, To bear with me for reprehending thee, For in a chair; his WIFE and a SERVITOR NERISSA.

But for thy care and honest men, our just censures Attend the true and humble subject to invent anything that I no more than we are here.

And here ye lie baiting of bombards, when Ye should do That in a well-hall ow'd cause.

- 上下文无关的文法 (Context-Free Grammar)
- 句法分析 (Parsing)
- n元模型 (n-gram)
- 分词 (Tokenization)
- 基于马尔科夫模型的文本生成

预告：基于统计的语言模型、信息检索、词表达、Transformer等

谢谢



北京大学
PEKING UNIVERSITY

